

HTML INJECTION

INTRODUCTION

Web applications often accept input from users and display it dynamically on web pages. If this input is not properly validated or sanitized, attackers can inject harmful HTML or script content, which can manipulate the appearance and functionality of the website. This vulnerability is known as **HTML Injection**. It is a common web security flaw that can lead to unauthorized data access, phishing attacks, website defacement, or even code execution on the client-side. Understanding how HTML Injection works, its types, how attackers exploit it, and how to prevent it is crucial for securing web applications.

1. What is HTML Injection?

HTML Injection is a type of security vulnerability where an attacker adds malicious HTML or JavaScript code to a web application's user interface. This happens when the application takes user input and embeds it into the HTML content of a page without proper validation or encoding. As a result, attackers can manipulate the page's layout, add deceptive links, display fake forms, or run malicious scripts that target other users visiting the site.

For example, if a search bar accepts user input and displays it directly on the page, an attacker could input something like `<a href='https://malicious.com'>Click Here</a>`. If the website renders this input without escaping, the attacker's content will appear on the page, misleading users.

2. Types of HTML Injection

There are mainly three types of HTML Injection:

1. **Stored (Persistent) HTML Injection**

The attacker's input is stored on the server, such as in a database or a comment section, and later displayed to other users. This is more dangerous because it can affect multiple users and persist over time.

2. Reflected (Non-Persistent) HTML Injection

The attacker's input is immediately reflected in the application's response without being stored. This is often seen in search bars or URL parameters and is commonly used in phishing and tricking users.

3. DOM-based HTML Injection

In this type, the vulnerability occurs in the client's browser through JavaScript manipulation of the Document Object Model (DOM). The input may come from URL fragments, query strings, or other sources that the script directly inserts into the page without proper sanitization.

3. How is HTML Injection Performed?

HTML Injection is performed by identifying vulnerable input points in a web application where user data is embedded into the page. The attacker submits input containing malicious HTML tags, such as links, images, or scripts.

Example payloads attackers might use:

- `<a href='https://malicious.com'>ClickMe</a>`
- `<script>alert('Hacked!')</script>`
- `<img src='x' onerror='alert(1)'`

These payloads exploit the application's failure to encode or escape characters like `<`, `>`, and `"` before inserting them into the page content. Once rendered by the browser, the injected content takes effect, potentially redirecting users, stealing cookies, or defacing the site.

4. How to Test Against HTML Injection?

Testing for HTML Injection involves attempting to inject HTML content and observing how the application handles it. Below are the steps:

1. **Identify Inputs** – Find fields such as search boxes, feedback forms, or URL parameters that accept user input.
2. **Inject Payloads** – Use HTML elements such as links or scripts.
Example:

`<a href='https://malicious.com'>ClickHere</a>` or
`<script>alert('Test')</script>`.

3. **Submit and Observe** – Check if the input is rendered as HTML or executed by the browser.
4. **Check for Error Messages** – Sometimes, the application throws errors that reveal improper input handling or backend queries, as seen in SQL-based injections.
5. **Use Tools** – Automated scanners like OWASP ZAP, Burp Suite, or Acunetix can help detect and analyze injection points.

5. How to Prevent HTML Injection?

Preventing HTML Injection requires careful handling of user inputs and outputs:

1. **Input Validation**
Restrict inputs to expected formats. For example, if a field should accept only alphanumeric characters, block all other symbols.
2. **Output Encoding**
Encode user input before embedding it in the page. Convert special characters like `<`, `>`, and `"` into their HTML entities (`<`, `>`, `"`;) so that the browser does not interpret them as code.
3. **Use Safe APIs and Frameworks**
Modern frameworks automatically escape outputs. Use libraries that handle encoding, such as React, Angular, or server-side templating engines.
4. **Limit Error Messages**
Avoid exposing detailed error messages to users. Instead, log errors internally while showing generic messages like "Invalid input."
5. **Content Security Policy (CSP)**
Implement CSP headers to restrict where scripts can be loaded from and prevent unauthorized content from executing in the browser.
6. **Regular Security Audits**
Conduct code reviews, penetration testing, and automated scanning to detect vulnerabilities before attackers exploit them.

CONCLUSION

HTML Injection is a serious web security issue that arises from improper handling of user inputs in web applications. By injecting malicious HTML or script content, attackers can manipulate web pages, steal sensitive information, or launch phishing attacks. Understanding the types, methods, and testing techniques is essential to identify vulnerabilities early. Applying input validation, output encoding, and modern security practices helps prevent such attacks and ensures that web applications remain safe for users. Continuous monitoring and security best practices are crucial in defending against evolving threats.

HTML INJECTION TESTING REPORT

Tested Target: <http://testphp.vulnweb.com/>

Vulnerability Found: HTML Injection (SQL Error Based Injection)

Description:

While testing the target web application, an HTML injection vulnerability was identified in the search functionality. The application failed to properly sanitize user input before processing it in the backend SQL query. By injecting HTML content into the search input, it caused the application to generate an error exposing sensitive database information.

Steps to Reproduce:

1. Open the target URL: <http://testphp.vulnweb.com/>.
2. Go to the search feature.
3. Enter the following input into the search field:
`<a href='https://youtube.com'>ClickMe</a>`

4. Submit the search request.
5. The application returned an SQL error message revealing details about the database query structure.
- 6.

Observed Result:

The error message displayed was:

Error: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '://youtube.com'>ClickMe', a.title) > 0 OR LOCATE('

This error confirms that the application embedded the HTML input into the SQL query without validation, allowing injection and exposing internal logic.

Impact:

- An attacker can exploit this vulnerability to:
 - Inject malicious scripts or content.
 - Leak database structure or sensitive information.
 - Conduct further attacks like SQL injection, session hijacking, or privilege escalation.
- The application's integrity and user data confidentiality are at risk.

Recommendations:

1. Sanitize and validate user inputs to ensure harmful content cannot be injected.
2. Use prepared statements or parameterized queries instead of concatenating user input into SQL statements.
3. Avoid displaying detailed error messages to users; log them securely instead.
4. Implement web application firewalls to detect and block malicious inputs.